ResumeMatch AI

Recruitment Matching Engine

Software Requirements Specification (SRS)

Purpose: Define the functional, non-functional, architectural, security, and deployment requirements for an AI-assisted recruitment matching platform.

Document	ResumeMatch AI – SRS
Version	1.0
Status	Baseline / Project Specification
Primary Users	Recruiters / Hiring Teams / Administrators
Technology	React, Node.js + Express, MongoDB, Python FastAPI, NLP, Vector Search
Deployment	Docker, Kubernetes, AWS EKS, AWS S3

Important product principle: ResumeMatch AI is a decision-support system. It assists recruiters in discovering and shortlisting candidates; it must not automatically reject candidates solely because of an AI score.

Table of Contents

1. Introduction

2. Overall Description

3. System Architecture and Technology Stack

4. Functional Requirements

5. AI / NLP Matching Requirements

6. Data Requirements

7. API Requirements

8. User Interface Requirements

9. Non-Functional Requirements

10. Security, Privacy and Responsible AI

11. Edge Cases and Failure Handling

12. Observability and Monitoring

13. Deployment and Infrastructure

14. Testing and Acceptance Criteria

15. Future Enhancements

1. Introduction

1.1 Purpose

This SRS defines the requirements for ResumeMatch AI, a recruitment matching engine that helps recruiters search and prioritize candidates using job requirements, resume information, NLP, semantic embeddings, and transparent matching evidence.

The system is intended to reduce dependence on exact keyword matching while keeping the recruiter in control of the final hiring decision.

1.2 Scope

- Accept a job description or structured job requirements.
- Accept candidate resumes in supported document formats.
- Parse and normalize resume content.
- Extract skills, education, experience, projects, certifications, and other relevant signals.
- Extract job requirements and distinguish required versus preferred criteria when possible.
- Generate semantic embeddings for job and candidate information.
- Retrieve and rank potentially relevant candidates using vector similarity and structured signals.
- Provide transparent evidence explaining why a candidate matched or did not match requirements.
- Allow recruiters to inspect candidate profiles and adjust search/filter criteria.
- Persist candidate, job, matching, and audit information securely.
- Provide monitoring and operational visibility for the AI and application services.

1.3 Intended Audience

- Recruiters and hiring teams using the matching interface.
- Administrators responsible for configuration, security, and monitoring.
- Backend and frontend developers.
- ML/NLP engineers responsible for extraction, embeddings, and ranking.
- QA engineers responsible for functional, integration, and AI evaluation testing.

1.4 Definitions

Term	Meaning
JD	Job Description / Job Requirements
Candidate	Person represented by a submitted resume/profile
Embedding	Numerical vector representation of text used for semantic similarity
Vector DB	Database/index optimized for similarity search over embeddings
Match Evidence	Human-readable reasons supporting a candidate match
Required Skill	A requirement explicitly or implicitly treated as necessary for the role
Preferred Skill	A useful but non-mandatory requirement

2. Overall Description

2.1 Product Perspective

ResumeMatch AI consists of a web client, an application/API layer, an AI/NLP service, persistent storage, vector search, caching, object storage, and observability infrastructure.

2.2 High-Level Workflow

Step	Process	Expected Output
1	Recruiter creates job / submits JD	Job record + normalized requirements
2	Candidate resumes uploaded	Stored resume files
3	Resume parsing	Extracted text and structured candidate data
4	NLP extraction	Skills, experience, education, entities, etc.
5	Embedding generation	Candidate and requirement vectors
6	Candidate retrieval	Relevant candidate set
7	Scoring/ranking	Ranked candidates + component scores
8	Evidence generation	Matched / missing requirements
9	Recruiter review	Shortlist, notes, actions, feedback

2.3 Assumptions

- Recruiters have permission to access the candidate data they search.
- Uploaded resumes are readable and in supported formats.
- The system may process resumes containing incomplete, inconsistent, or differently formatted information.
- AI scores are advisory and are not the sole basis for employment decisions.
- The vector database can be implemented using a production-ready vector-search technology compatible with the chosen architecture.

2.4 Constraints

- The system must protect sensitive candidate information.
- AI explanations must be traceable to extracted information or explicit matching rules.
- Long-running parsing/embedding tasks should not block normal recruiter API requests.
- The system should scale independently across web/API and AI workloads.

3. System Architecture and Technology Stack

3.1 Proposed Logical Architecture

The system is divided into the following logical components:

Layer	Technology / Component	Responsibility
Frontend	React	Recruiter UI, search, candidate review, evidence display
Application API	Node.js + Express	Authentication, jobs, candidates, search, workflow orchestration
AI Service	Python + FastAPI	Resume parsing, NLP, embeddings, scoring support
NLP	spaCy	Entity extraction, text processing, normalization
Embeddings	Sentence Transformers	Semantic representation of job/candidate text
ML utilities	scikit-learn	Similarity/scoring utilities, evaluation, optional classifiers
Primary DB	MongoDB	Users, jobs, candidates, extracted metadata, match records
Vector DB	Vector database	Embedding storage and similarity retrieval
Cache / Queue support	Redis	Caching, rate limiting, job state / asynchronous workflow support
Object storage	AWS S3	Original resumes and related files
Containerization	Docker	Consistent packaging and local/CI execution
Orchestration	Kubernetes / AWS EKS	Scalable deployment and service management
Monitoring	Prometheus + Grafana	Metrics, dashboards, operational monitoring

3.2 Service Interaction

- React communicates with Node.js/Express APIs over HTTPS.
- Node.js/Express calls the FastAPI AI service for parsing, extraction, embedding, and matching operations.
- MongoDB stores application and structured candidate/job data.
- S3 stores original uploaded resume files; the database stores references and metadata.
- Vector search stores embeddings and returns semantically similar candidates or requirements.
- Redis supports caching and asynchronous processing state where required.
- Prometheus collects service metrics and Grafana presents operational dashboards.

4. Functional Requirements

4.1 Authentication and Authorization

- FR-AUTH-01: The system shall allow authorized recruiters to sign in.
- FR-AUTH-02: The system shall enforce role-based access to recruiter and administrator functions.
- FR-AUTH-03: The system shall reject unauthorized access to candidate and job data.
- FR-AUTH-04: The system shall maintain audit information for security-sensitive actions.

4.2 Job Management

- FR-JOB-01: A recruiter shall be able to create a job using a job title, description, skills, experience, education, location/work mode, and optional preferences.
- FR-JOB-02: The system shall parse free-text job descriptions into structured requirements where possible.
- FR-JOB-03: The recruiter shall be able to edit extracted requirements before matching.
- FR-JOB-04: Requirements shall support required and preferred categories.
- FR-JOB-05: The system shall maintain job status such as draft, active, paused, and closed.

4.3 Resume / Candidate Management

- FR-CAN-01: The system shall accept resumes in configured supported formats.
- FR-CAN-02: Uploaded files shall be stored securely and associated with candidate metadata.
- FR-CAN-03: The system shall extract text from resumes and normalize it for NLP processing.
- FR-CAN-04: The system shall extract relevant candidate attributes such as skills, job history, education, certifications, projects, and technologies when available.
- FR-CAN-05: The system shall preserve the original resume and extracted representation separately.
- FR-CAN-06: The system shall indicate parsing failures or low-confidence extraction rather than silently inventing information.

4.4 Candidate Search and Matching

- FR-MATCH-01: The recruiter shall be able to initiate matching for a selected job.
- FR-MATCH-02: The system shall retrieve semantically relevant candidates rather than relying only on exact keyword overlap.
- FR-MATCH-03: The ranking engine shall combine semantic similarity with structured matching signals where available.
- FR-MATCH-04: The system shall support recruiter filters such as skills, experience, education, location/work mode, and other configured attributes.
- FR-MATCH-05: The system shall return ranked candidates with an overall match score.
- FR-MATCH-06: The system shall show component-level or requirement-level evidence behind the score.
- FR-MATCH-07: The system shall not automatically reject a candidate solely because of an AI match score.

4.5 Evidence and Recruiter Actions

- FR-EVID-01: The system shall display matched skills/requirements.
- FR-EVID-02: The system shall display missing or unmatched requirements when determinable.
- FR-EVID-03: The system shall identify relevant experience or projects supporting a match where evidence exists.
- FR-EVID-04: The recruiter shall be able to open the underlying candidate resume/profile for verification.

- FR-EVID-05: The recruiter shall be able to shortlist, save, add notes, or change candidate status.
- FR-EVID-06: Recruiter actions shall be stored for audit and future evaluation where permitted.

5. AI / NLP Matching Requirements

5.1 Resume Processing Pipeline

- The pipeline shall ingest resume text after file extraction.
- spaCy shall be used for configurable linguistic processing and entity extraction.
- Skills and technologies shall be normalized to reduce variation such as abbreviations and synonymous terms.
- Experience shall be represented using structured fields where extraction is reliable.
- The pipeline shall retain source text spans or references for important extracted evidence whenever practical.

5.2 Semantic Embeddings

- Sentence Transformers shall generate embeddings for job requirements and candidate information.
- Embeddings shall be versioned so that changes to the model can be tracked.
- The vector store shall support similarity search and metadata filtering.
- The system shall avoid comparing unrelated text blocks when a more precise representation is available.

5.3 Matching / Ranking Model

A configurable ranking strategy is recommended so that semantic similarity is not the only signal. A conceptual score can be represented as:

Overall Match Score = w■ × Semantic Similarity + w■ × Skill Match + w■ × Experience Match + w■ × Education/Certification Match + w■ × Other Configured Signals

Weights shall be configurable and documented. The final score must not be presented as a probability of hiring or a guarantee of suitability.

5.4 Explainability Requirements

- The system shall show which job requirements were matched and what candidate evidence supports them.
- The system shall show missing or uncertain requirements separately from positively matched requirements.
- The explanation should distinguish 'not found in resume' from 'confirmed absent' where the system cannot reliably establish absence.
- AI-generated explanations shall be grounded in extracted candidate/job data and must not invent qualifications.
- Low-confidence extraction shall be surfaced to the recruiter where it can affect matching.

6. Data Requirements

6.1 Core Entities

Entity	Representative Fields
User	userId, name, email, role, status, createdAt
Job	jobId, title, description, requirements, preferences, status, createdBy, timestamps
Candidate	candidateId, profile metadata, resume references, extracted profile, status, timestamps
Resume	resumeId, candidateId, S3 key, file type, parse status, text reference, timestamps
Skill	skillId/name, normalizedName, aliases, category
Embedding	embeddingId, entityId, modelVersion, vectorRef, createdAt
Match	matchId, jobId, candidateId, score, components, evidence, modelVersion, timestamp
Audit Log	eventId, actorId, action, resource, timestamp, metadata

6.2 Data Quality

- Duplicate candidate and duplicate resume handling shall be defined.
- Malformed or empty resumes shall be flagged.
- Missing fields shall remain missing rather than being filled with guessed values.
- Normalization rules shall be versioned when they materially affect matching.

6.3 Retention

Candidate documents, extracted data, embeddings, and audit records shall have configurable retention policies appropriate to organizational requirements and applicable privacy obligations.

7. API Requirements

The Node.js/Express service should expose REST-style APIs. Exact route names may be finalized during implementation.

Area	Example Endpoint	Purpose
Auth	POST /api/auth/login	Authenticate recruiter
Jobs	POST /api/jobs	Create job
Jobs	GET /api/jobs/:id	Retrieve job
Jobs	PUT /api/jobs/:id	Update job
Candidates	POST /api/candidates	Create/import candidate
Resumes	POST /api/candidates/:id/resumes	Upload resume
Search	POST /api/jobs/:id/match	Start matching
Results	GET /api/jobs/:id/matches	Retrieve ranked matches
Evidence	GET /api/matches/:id/evidence	Retrieve matching evidence
Actions	PATCH /api/candidates/:id/status	Update recruiter status
AI	POST /ai/parse-resume	Parse resume
AI	POST /ai/extract-profile	Extract candidate data
AI	POST /ai/embed	Generate embeddings

7.1 API Behavior

- APIs shall validate request payloads and return consistent error responses.
- Long-running AI operations should use asynchronous jobs where appropriate.
- API responses shall include stable identifiers and timestamps.
- Sensitive data shall not be exposed in logs or unnecessary API responses.

8. User Interface Requirements

8.1 Recruiter Dashboard

- Show active jobs and high-level candidate matching activity.
- Provide clear entry points for creating a job and uploading candidates.

8.2 Job Creation / Requirement Review

- Provide a text area or upload mechanism for the job description.
- Display extracted skills and requirements for recruiter confirmation/editing.
- Clearly distinguish required and preferred requirements.

8.3 Candidate Ranking Screen

- Show ranked candidates with an understandable match score.
- Provide filters and sorting controls.
- Show concise matched-skill and missing-requirement summaries.
- Allow opening detailed evidence without requiring the recruiter to understand the AI implementation.

8.4 Candidate Detail

- Display candidate profile, resume preview/download according to permissions, extracted skills, experience, and match evidence.
- Show uncertainty or missing information where relevant.

8.5 Accessibility and UX

- Use readable labels and consistent navigation.
- Provide clear loading, empty, error, and success states.
- Avoid presenting AI scores as definitive hiring decisions.

9. Non-Functional Requirements

Category	Requirement
Performance	Typical recruiter API requests should respond quickly; long-running parsing/matching work should be asynchronous where necessary.
Scalability	Services shall scale horizontally in Kubernetes; AI workers should scale independently from API servers.
Availability	Production deployment should use health checks, restart policies, and redundant service instances where required.
Reliability	Failures in parsing or individual candidate processing should not fail an entire batch.
Maintainability	Backend, AI service, and frontend should have clear module boundaries and documented interfaces.
Observability	Expose metrics, logs, and health endpoints for critical services.
Security	Use encrypted transport, access controls, secure file handling, and least-privilege permissions.
Privacy	Candidate information shall be treated as sensitive personal data and handled according to applicable policies/laws.
Explainability	Match results shall include evidence and model/version information sufficient for internal review.
Portability	Dockerized services should be runnable locally and in Kubernetes.

10. Security, Privacy and Responsible AI

10.1 Security

- All production API communication shall use HTTPS/TLS.
- Authentication tokens/session credentials shall be protected using secure practices.
- Passwords, secrets, API keys, and cloud credentials shall not be stored in source code.
- S3 objects shall use controlled access rather than public exposure.
- MongoDB and vector database access shall require authentication and network controls.
- Upload endpoints shall validate file type, size, and content handling rules.
- Audit logs shall record security-relevant actions without unnecessarily storing sensitive resume contents.

10.2 Responsible AI

- The system is an assistive recruitment tool, not an autonomous hiring decision-maker.
- The ranking system shall not be intentionally based on protected characteristics or proxy features that create unjustified discrimination.
- Recruiters shall be able to inspect evidence and override the ranking through human judgment.
- The product should support periodic evaluation for disparate performance, extraction errors, and ranking quality.
- The UI shall clearly communicate that a low score may reflect missing information rather than proof of lack of qualification.

10.3 Privacy

- Access to candidate resumes and extracted information shall be limited to authorized users.
- Data retention and deletion workflows shall be configurable.
- Candidate deletion shall define behavior for associated embeddings, cached data, and stored files.

11. Edge Cases and Failure Handling

ID	Edge Case	Expected Behavior
EC-01	Empty resume	Reject or flag as unprocessable; do not generate a misleading match.
EC-02	Corrupted file	Show upload/processing failure and allow re-upload.
EC-03	Unsupported format	Reject with supported-format guidance.
EC-04	Scanned/image-only resume	If OCR is supported, process it; otherwise flag as requiring manual review.
EC-05	Multiple resumes for one candidate	Maintain versioned documents and define the active/latest resume.
EC-06	Duplicate resumes	Detect likely duplicates and avoid double-counting candidates where configured.
EC-07	Skill synonym	Normalize aliases/synonyms before matching when supported.
EC-08	Missing information	Show 'not found/unknown' rather than treating it as confirmed absence.
EC-09	Very long resume	Apply configured size/token limits and preserve processing status.
EC-10	AI service unavailable	Queue/retry safely and show processing status; do not silently return stale results.
EC-11	Vector DB unavailable	Fail gracefully, retry if safe, and expose service health state.
EC-12	No strong candidates	Show low-confidence/low-score results with transparent evidence rather than automatically rejecting all candidates.
EC-13	Conflicting resume information	Preserve source evidence and flag ambiguity where possible.
EC-14	Model version changes	Store model version with generated embeddings and match records.
EC-15	Prompt/text injection in resume	Treat resume content as untrusted data; do not allow resume text to alter system instructions or security behavior.

12. Observability and Monitoring

12.1 Metrics

- API request count, latency, error rate, and status codes.
- Resume upload and parsing success/failure rates.
- AI processing latency and queue depth.
- Embedding generation latency and failures.
- Vector search latency and result counts.
- Matching job duration and batch failure rates.
- Redis/cache hit rate where applicable.
- CPU, memory, pod restarts, and resource utilization.

12.2 Prometheus and Grafana

- Prometheus shall collect application and infrastructure metrics.
- Grafana dashboards shall expose API health, AI processing, matching workload, and infrastructure status.
- Alert thresholds shall be defined for sustained errors, high latency, resource exhaustion, and service unavailability.

13. Deployment and Infrastructure

13.1 Local Development

- Docker Compose or equivalent local orchestration may be used for frontend, API, AI service, MongoDB, Redis, and vector database.
- Environment-specific configuration shall be externalized.

13.2 Production

- Container images shall be built reproducibly and scanned for vulnerabilities.
- Kubernetes on AWS EKS shall run scalable application and AI workloads.
- AWS S3 shall store resume objects with private access controls.
- Secrets shall be managed through a secure secret-management mechanism rather than committed to Git.
- Health checks/readiness/liveness probes shall be configured for deployable services.

13.3 CI/CD

- Automated tests should run before deployment.
- Container images should be versioned.
- Production deployment should support rollback to a known-good version.

14. Testing and Acceptance Criteria

14.1 Testing Levels

- Unit testing for parsing, normalization, scoring, API validation, and UI components.
- Integration testing for Node.js ↔ FastAPI ↔ MongoDB/vector DB/Redis flows.
- End-to-end testing for recruiter workflows.
- Performance/load testing for candidate search and batch matching.
- Security testing for authentication, authorization, uploads, and API exposure.
- AI evaluation for extraction accuracy, ranking quality, semantic matching, and explanation grounding.

14.2 Core Acceptance Criteria

AC	Acceptance Criterion
AC-01	Recruiter can create a job and review extracted requirements.
AC-02	Recruiter can upload supported resumes and see processing status.
AC-03	The system extracts meaningful candidate information for valid resumes.
AC-04	A job can be matched against a candidate pool using semantic retrieval.
AC-05	Results are ranked and include understandable matching evidence.
AC-06	Missing information is not falsely presented as confirmed absence.
AC-07	The system does not automatically reject candidates solely because of an AI score.
AC-08	A failed candidate in a batch does not prevent other candidates from being processed.
AC-09	Unauthorized users cannot access protected candidate information.
AC-10	Operational metrics and health status are available to administrators.

15. Future Enhancements

- Recruiter feedback loops for ranking improvement.
- Candidate-job matching calibration and offline evaluation dashboards.
- Multilingual resume support.
- OCR integration for scanned resumes.
- Skill ontology / knowledge graph for richer relationships between technologies and roles.
- Hybrid ranking using learning-to-rank models after sufficient labeled feedback is available.
- Advanced duplicate detection and candidate profile merging.
- Fine-grained recruiter permissions and organization-level tenancy.
- Model monitoring for drift and fairness-related performance indicators.

Appendix A – Suggested Initial Project Modules

Module	Suggested Responsibilities
frontend/	React application, pages, components, API client, state management
backend/	Express routes, controllers, authentication, database models, orchestration
ai-service/	FastAPI endpoints, spaCy pipeline, embedding generation, scoring utilities
workers/	Asynchronous resume parsing/matching jobs if separated from AI service
infra/	Docker, Kubernetes manifests/Helm, AWS/EKS configuration
monitoring/	Prometheus configuration, Grafana dashboards/alerts
tests/	Unit, integration, E2E, AI evaluation datasets and tests

End of SRS – ResumeMatch AI